

SC 663: Optimization and Inference in Sparse Network Models

Homework 1

Shrivardhan Kondekar
Centre of Systems and Control
IIT Bombay

Due Date: August 25, 2026

Problem 1

For $x = (x_1, \dots, x_n) \in \mathbb{R}^n$ and $1 \leq p < \infty$, define $\|x\|_p \triangleq \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$ (expressed slightly different than the book) and $\|x\|_\infty \triangleq \max_{1 \leq i \leq n} |x_i|$. Let $\|x\|_0$ denote the number of nonzero entries of x .

1. Prove that

$$\|x\|_\infty = \lim_{p \rightarrow \infty} \|x\|_p.$$

Solution.

Let $x = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$. By definition,

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}, \quad \|x\|_\infty = \max_{1 \leq i \leq n} |x_i|.$$

Let

$$M = \max_{1 \leq i \leq n} |x_i| = \|x\|_\infty.$$

For the case of M being infinity, the equality is trivial. So we assume that M is finite. Since M is the maximum of the components, we have

$$|x_i|^p \leq M^p$$

for every i . Therefore,

$$\sum_{i=1}^n |x_i|^p \leq nM^p,$$

and hence

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p} \leq n^{1/p} M.$$

On the other hand, there exists some $j \in \{1, \dots, n\}$ such that

$$|x_j| = M.$$

Thus,

$$\sum_{i=1}^n |x_i|^p \geq |x_j|^p = M^p,$$

which gives

$$\|x\|_p \geq M.$$

Combining the two inequalities, we obtain

$$M \leq \|x\|_p \leq n^{1/p} M.$$

Since

$$\lim_{p \rightarrow \infty} n^{1/p} = 1,$$

Therefore

$$M \leq \lim_{p \rightarrow \infty} \|x\|_p \leq M$$

So, from the sandwich theorem and using the fact that $M = \|x\|_\infty$, we conclude that

Since $M = \|x\|_\infty$, we conclude that

$$\boxed{\lim_{p \rightarrow \infty} \|x\|_p = \|x\|_\infty}.$$

□

2. Let $0 < p < 1$ in $\|x\|_p$. Show

$$\lim_{p \rightarrow 0^+} \|x\|_p^p = \|x\|_0.$$

Solution.

Let $x = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$. By definition,

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p},$$

and hence

$$\|x\|_p^p = \sum_{i=1}^n |x_i|^p.$$

For each i , if $x_i \neq 0$, then

$$|x_i|^p = e^{p \ln |x_i|} \longrightarrow 1 \quad \text{as } p \rightarrow 0^+.$$

If $x_i = 0$, then

$$|x_i|^p = 0$$

for every $p > 0$. Therefore,

$$\lim_{p \rightarrow 0^+} |x_i|^p = \begin{cases} 1, & x_i \neq 0, \\ 0, & x_i = 0. \end{cases}$$

Since the sum contains finitely many terms,

$$\begin{aligned} \lim_{p \rightarrow 0^+} \|x\|_p^p &= \lim_{p \rightarrow 0^+} \sum_{i=1}^n |x_i|^p \\ &= \sum_{i=1}^n \lim_{p \rightarrow 0^+} |x_i|^p \\ &= \sum_{i=1}^n \mathbf{1}_{\{x_i \neq 0\}} \\ &= \|x\|_0. \end{aligned}$$

Thus,

$$\boxed{\lim_{p \rightarrow 0^+} \|x\|_p^p = \|x\|_0}.$$

□

3. Prove that $\|x + y\|_0 \leq \|x\|_0 + \|y\|_0$. When does equality hold?

Solution.

For $x = (x_1, \dots, x_n)$ and $y = (y_1, \dots, y_n)$, we have

$$\|x\|_0 = \sum_{i=1}^n \mathbf{1}_{\{x_i \neq 0\}}, \quad \|y\|_0 = \sum_{i=1}^n \mathbf{1}_{\{y_i \neq 0\}}.$$

For each i , if $x_i + y_i \neq 0$, then at least one of x_i and y_i is nonzero. Hence

$$\mathbf{1}_{\{x_i + y_i \neq 0\}} \leq \mathbf{1}_{\{x_i \neq 0\}} + \mathbf{1}_{\{y_i \neq 0\}}.$$

Summing over $i = 1, \dots, n$, we obtain

$$\|x + y\|_0 \leq \|x\|_0 + \|y\|_0.$$

Thus,

$$\boxed{\|x + y\|_0 \leq \|x\|_0 + \|y\|_0.}$$

□

Equality holds if and only if

$$\mathbf{1}_{\{x_i + y_i \neq 0\}} = \mathbf{1}_{\{x_i \neq 0\}} + \mathbf{1}_{\{y_i \neq 0\}}$$

for every i . This is equivalent to

$$x_i \neq 0, \quad y_i \neq 0 \quad \implies \quad x_i + y_i \neq 0,$$

i.e., there is no index i for which

$$x_i \neq 0, \quad y_i \neq 0, \quad x_i + y_i = 0.$$

Problem 2

Exercise 2.13. For the optimization problem (2.38) of size m , compute the number of iterations at the worst case where the optimal solution x^* has its ℓ^0 norm as $\|x^*\|_0 = m$. Then, let $m = 100$. Suppose that you can use a supercomputer that can do one iteration of the exhaustive search algorithm in 10^{-15} seconds. Compute the total time needed to do the exhaustive search at the worst case.

Solution.

In the worst case,

$$\|x^*\|_0 = m.$$

An exhaustive search must consider every possible combination of nonzero components. Since each of the m components can either be zero or nonzero, the total number of possible combinations is

$$\sum_{k=0}^m \binom{m}{k} = 2^m.$$

Therefore, the number of iterations required in the worst case is

$$\boxed{2^m}.$$

For $m = 100$, the number of iterations is

$$2^{100} = 1.2676506002282294 \times 10^{30}.$$

Since each iteration takes 10^{-15} seconds, the total time required is

$$T = 2^{100} \times 10^{-15} \text{ seconds.}$$

Hence,

$$T \approx 1.26765 \times 10^{15} \text{ seconds.}$$

Converting this into years,

$$T = \frac{1.26765 \times 10^{15}}{60 \times 60 \times 24 \times 365} \text{ years} \\ \approx 4.02 \times 10^7 \text{ years.}$$

Thus, even with a supercomputer capable of performing one iteration in 10^{-15} seconds, the exhaustive search would take approximately

$$\boxed{4.02 \times 10^7 \text{ years}}$$

Problem 3

Let $x^* \in \mathbb{R}^n$ be a sparsest solution of $\Phi x = y$, where $\Phi \in \mathbb{R}^{m \times n}$ with $m < n$. Suppose that $\text{supp}(x^*) = \{i_1, \dots, i_s\}$. Writing $\Phi = [\phi_1, \dots, \phi_n]$, show that the columns $\phi_{i_1}, \dots, \phi_{i_s}$ are linearly independent.

Solution.

Let

$$\text{supp}(x^*) = \{i_1, \dots, i_s\},$$

and suppose, for contradiction, that the columns

$$\phi_{i_1}, \dots, \phi_{i_s}$$

are linearly dependent. Then there exists a nonzero vector

$$d \in \mathbb{R}^n$$

such that

$$\text{supp}(d) \subseteq \{i_1, \dots, i_s\}, \quad \Phi d = 0.$$

Consider the vector

$$x^* + \alpha d.$$

Since $\Phi d = 0$,

$$\Phi(x^* + \alpha d) = \Phi x^* + \alpha \Phi d = y.$$

Thus, $x^* + \alpha d$ is also a solution of

$$\Phi x = y$$

for every $\alpha \in \mathbb{R}$.

Since $d \neq 0$, choose an index $j \in \text{supp}(d)$. Then

$$d_j \neq 0.$$

Choose

$$\alpha = -\frac{x_j^*}{d_j}.$$

For this choice,

$$x_j^* + \alpha d_j = 0.$$

Moreover, since

$$\text{supp}(d) \subseteq \text{supp}(x^*),$$

we have

$$\text{supp}(x^* + \alpha d) \subseteq \text{supp}(x^*).$$

Therefore, at least one nonzero component of x^* has been eliminated, while no new nonzero component can appear. Hence

$$\|x^* + \alpha d\|_0 < \|x^*\|_0.$$

But $x^* + \alpha d$ is a solution of $\Phi x = y$, contradicting the assumption that x^* is a sparsest solution.

Therefore, the columns

$$\boxed{\phi_{i_1}, \dots, \phi_{i_s}}$$

must be linearly independent.

□

Problem 4

(Polynomial curve fitting in Ch 3 of the SMSC book). Define the $n - 1$ -th order polynomial:

$$y = f(t) = a_{n-1}t^{n-1} + \cdots + a_1t + a_0.$$

Define $a = [a_0, a_1, \dots, a_{n-1}]^\top$ and $y = [y_1, \dots, y_m]^\top$. Consider the curve-fitting equations as $y = \Phi a$, where $\Phi \in \mathbb{R}^{m \times n}$ is the corresponding Vandermonde matrix.

1. Let $\Psi \in \mathbb{R}^{m \times n}$. Derive the solution of the weighted regularized least-squares problem

$$\min_x \frac{1}{2} \|\Phi x - y\|_2^2 + \frac{\lambda}{2} \|\Psi x\|_2^2, \quad \lambda > 0.$$

2. Describe the behavior of the regularized solution as $\lambda \rightarrow 0$ and as $\lambda \rightarrow \infty$.
3. Show that, for any $\Phi \in \mathbb{R}^{m \times n}$ and $\lambda > 0$, the matrices

$$\lambda I + \Phi \Phi^\top \quad \text{and} \quad \lambda I + \Phi^\top \Phi$$

are invertible. Prove that

$$\Phi^\top (\lambda I + \Phi \Phi^\top)^{-1} = (\lambda I + \Phi^\top \Phi)^{-1} \Phi^\top.$$

Solution.

1. The objective function is

$$J(x) = \frac{1}{2} \|\Phi x - y\|_2^2 + \frac{\lambda}{2} \|\Psi x\|_2^2.$$

Expanding the two terms gives

$$J(x) = \frac{1}{2} (\Phi x - y)^\top (\Phi x - y) + \frac{\lambda}{2} (\Psi x)^\top (\Psi x).$$

Hence,

$$J(x) = \frac{1}{2} x^\top \Phi^\top \Phi x - x^\top \Phi^\top y + \frac{1}{2} y^\top y + \frac{\lambda}{2} x^\top \Psi^\top \Psi x \quad (\because x^\top \Phi^\top y = y^\top \Phi x)$$

Taking the gradient with respect to x ,

$$\nabla J(x) = \Phi^\top \Phi x - \Phi^\top y + \lambda \Psi^\top \Psi x.$$

At the minimizer x^* ,

$$\nabla J(x^*) = 0.$$

Therefore,

$$(\Phi^\top \Phi + \lambda \Psi^\top \Psi) x^* = \Phi^\top y.$$

Thus, provided the matrix is invertible,

$$x^* = (\Phi^\top \Phi + \lambda \Psi^\top \Psi)^{-1} \Phi^\top y.$$

2. From

$$x^* = (\Phi^\top \Phi + \lambda \Psi^\top \Psi)^{-1} \Phi^\top y,$$

as $\lambda \rightarrow 0$, the regularization term vanishes. Therefore,

$$x^* \longrightarrow (\Phi^\top \Phi)^{-1} \Phi^\top y,$$

provided $\Phi^\top \Phi$ is invertible. This is the ordinary least-squares solution.

As $\lambda \rightarrow \infty$, the regularization term dominates the objective. Consequently, the solution tends toward a vector minimizing

$$\|\Psi x\|_2^2.$$

In particular, if Ψ is invertible, then

$$x^* \rightarrow 0.$$

3. Let

$$A = \lambda I + \Phi \Phi^\top.$$

For any nonzero $z \in \mathbb{R}^m$,

$$\begin{aligned} z^\top A z &= z^\top (\lambda I + \Phi \Phi^\top) z \\ &= \lambda \|z\|_2^2 + \|\Phi^\top z\|_2^2. \end{aligned}$$

Since $\lambda > 0$ and $z \neq 0$,

$$\lambda \|z\|_2^2 + \|\Phi^\top z\|_2^2 > 0.$$

Hence,

$$\lambda I + \Phi \Phi^\top$$

is positive definite and therefore invertible.

Similarly, for any nonzero $z \in \mathbb{R}^n$,

$$\begin{aligned} z^\top (\lambda I + \Phi^\top \Phi) z &= \lambda \|z\|_2^2 + \|\Phi z\|_2^2 \\ &> 0. \end{aligned}$$

Therefore,

$$\lambda I + \Phi^\top \Phi$$

is also positive definite and hence invertible.

Finally, observe that

$$(\lambda I + \Phi^\top \Phi) \Phi^\top = \lambda \Phi^\top + \Phi^\top \Phi \Phi^\top$$

and

$$\Phi^\top (\lambda I + \Phi \Phi^\top) = \lambda \Phi^\top + \Phi^\top \Phi \Phi^\top.$$

Thus,

$$(\lambda I + \Phi^\top \Phi) \Phi^\top = \Phi^\top (\lambda I + \Phi \Phi^\top).$$

Multiplying from the left by $(\lambda I + \Phi^\top \Phi)^{-1}$ and from the right by $(\lambda I + \Phi \Phi^\top)^{-1}$, we obtain

$$\boxed{\Phi^\top (\lambda I + \Phi \Phi^\top)^{-1} = (\lambda I + \Phi^\top \Phi)^{-1} \Phi^\top.}$$

□

Problem 5

(Computational exercise: polynomial fitting). Consider the following temperature measurements over a 24-hour period:

t	1	2	3	4	5	6	7	8	9	10	11	12
y	75	77	76	73	69	68	63	59	57	55	54	52
t	13	14	15	16	17	18	19	20	21	22	23	24
y	50	50	49	49	49	50	54	56	59	63	67	72

To avoid numerical difficulties associated with powers of t , first normalize the sampling instants as

$$s_i = \frac{t_i - 12.5}{11.5}, \quad i = 1, \dots, 24.$$

Consider a polynomial of order 23; that is $n - 1 = 23$. Using CVX (see Edition 1 of SMSC for Matlab and Edition 2 for Python), perform the following experiments.

1. Compute the interpolating polynomial, the ridge polynomial, and the sparse polynomial. For the latter two, use several values of $\lambda > 0$.
2. For representative values of λ , plot the temperature measurements together with the three fitted curves. Also plot the corresponding coefficient vectors x in another plot to visualize sparsity.

3. For a fitted coefficient vector x , define the relative fitting error as

$$E(x) = \frac{\|\Phi x - y\|_2}{\|y\|_2}.$$

Compare ridge and ℓ_1 regularization in terms of fitting error and sparsity of x as λ varies. Briefly explain your observations.

Code for (1)

```

1  import numpy as np
2  import cvxpy as cp
3
4  # Temperature measurements
5  t = np.arange(1, 25)
6  y = np.array([75, 77, 76, 73, 69, 68, 63, 59, 57, 55, 54, 52, 50, 50, 49, 49,
7              49, 50, 54, 56, 59, 63, 67, 72], dtype=float)
8
9  # Normalize the sampling instants
10 s = (t - 12.5) / 11.5
11
12 # Construct the Vandermonde matrix
13 n = 24
14 Phi = np.vander(s, N=n, increasing=True)
15
16 # Interpolating polynomial
17 x_interp = np.linalg.solve(Phi, y)
18 interp_error = np.linalg.norm(Phi @ x_interp - y)
19
20 print("INTERPOLATING POLYNOMIAL")
21 print("Coefficient vector:")
22 print(x_interp)
23 print(f"\nFitting residual ||Phi x - y||_2 = {interp_error:.6e}")
24
25 # Values of lambda
26 lambdas = [1e-4, 1e-2, 1e-1, 1, 10, 100]
27
28 # Storage for coefficient vectors
29 ridge_coefficients = {}
30 sparse_coefficients = {}
31
32 # Tolerance for determining whether a coefficient is zero
33 tol = 1e-6
34
35 # Ridge polynomial
36 for lam in lambdas:
37     x = cp.Variable(n)
38     objective = cp.Minimize(0.5 * cp.sum_squares(Phi @ x - y) + 0.5 * lam * cp.
39                             sum_squares(x))
40     problem = cp.Problem(objective)
41     problem.solve()
42
43     if x.value is None:
44         print(f"\nRidge, lambda = {lam}: solver failed")
45         continue
46
47     ridge_coefficients[lam] = x.value.copy()
48     residual = np.linalg.norm(Phi @ x.value - y)
49     l2_norm = np.linalg.norm(x.value)
50     l1_norm = np.linalg.norm(x.value, 1)
51     sparsity = np.sum(np.abs(x.value) > tol)
52
53     print(f"\nRidge, lambda = {lam}")
54     print(f"Status          : {problem.status}")
55     print(f"Residual          : {residual:.6e}")
56     print(f"||x||_1           : {l1_norm:.6e}")
57     print(f"||x||_2           : {l2_norm:.6e}")

```

```

56     print(f"||x||_0      : {sparsity}")
57     print("Coefficient vector:")
58     print(x.value)
59
60     # Sparse polynomial
61     for lam in lambdas:
62         x = cp.Variable(n)
63         objective = cp.Minimize(0.5 * cp.sum_squares(Phi @ x - y) + lam * cp.norm1(x
64             ))
65         problem = cp.Problem(objective)
66         problem.solve()
67
68         if x.value is None:
69             print(f"\nSparse, lambda = {lam}: solver failed")
70             continue
71
72         sparse_coefficients[lam] = x.value.copy()
73         residual = np.linalg.norm(Phi @ x.value - y)
74         l2_norm = np.linalg.norm(x.value)
75         l1_norm = np.linalg.norm(x.value, 1)
76         sparsity = np.sum(np.abs(x.value) > tol)
77
78         print(f"\nSparse, lambda = {lam}")
79         print(f"Status      : {problem.status}")
80         print(f"Residual    : {residual:.6e}")
81         print(f"||x||_1     : {l1_norm:.6e}")
82         print(f"||x||_2     : {l2_norm:.6e}")
83         print(f"||x||_0     : {sparsity}")
84         print("Coefficient vector:")
85         print(x.value)
86
87     # Summary table
88     print("\nSUMMARY")
89     print(f"{'Lambda':>10} {'Model':>10} {'Residual':>15} {'||x||_1':>15} {'||x||_2'
90         ':>15} {'||x||_0':>10}")
91     print("-" * 80)
92
93     for lam in lambdas:
94         if lam in ridge_coefficients:
95             x = ridge_coefficients[lam]
96             residual = np.linalg.norm(Phi @ x - y)
97             l1_norm = np.linalg.norm(x, 1)
98             l2_norm = np.linalg.norm(x)
99             sparsity = np.sum(np.abs(x) > tol)
100             print(f"{lam:10.1e} {'Ridge':>10} {residual:15.6e} {l1_norm:15.6e} {
101                 l2_norm:15.6e} {sparsity:10d}")
102
103         if lam in sparse_coefficients:
104             x = sparse_coefficients[lam]
105             residual = np.linalg.norm(Phi @ x - y)
106             l1_norm = np.linalg.norm(x, 1)
107             l2_norm = np.linalg.norm(x)
108             sparsity = np.sum(np.abs(x) > tol)
109             print(f"{lam:10.1e} {'Sparse':>10} {residual:15.6e} {l1_norm:15.6e} {
110                 l2_norm:15.6e} {sparsity:10d}")

```

Solution for (1)

The temperature measurements are given at the sampling instants

$$t = 1, 2, \dots, 24,$$

with corresponding measurements

$$y = [75, 77, 76, 73, 69, 68, 63, 59, 57, 55, 54, 52, 50, 50, 49, 49, 49, 50, 54, 56, 59, 63, 67, 72]^T.$$

We normalize the sampling instants according to

$$s_i = \frac{t_i - 12.5}{11.5},$$

so that $s_i \in [-1, 1]$. The Vandermonde matrix is then constructed as

$$\Phi = \begin{bmatrix} 1 & s_1 & s_1^2 & \cdots & s_1^{23} \\ 1 & s_2 & s_2^2 & \cdots & s_2^{23} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & s_{24} & s_{24}^2 & \cdots & s_{24}^{23} \end{bmatrix}.$$

The interpolating polynomial is obtained by solving

$$\Phi x = y.$$

The resulting fitting residual is

$$\|\Phi x - y\|_2 = 3.457423 \times 10^{-7},$$

which is effectively zero.

For the regularized polynomial, we consider the values

$$\lambda \in \{10^{-4}, 10^{-2}, 10^{-1}, 1, 10, 100\}.$$

The ridge polynomial is obtained by solving

$$\min_x \frac{1}{2} \|\Phi x - y\|_2^2 + \frac{\lambda}{2} \|x\|_2^2,$$

while the sparse polynomial is obtained from

$$\min_x \frac{1}{2} \|\Phi x - y\|_2^2 + \lambda \|x\|_1.$$

The numerical results are summarized in Table 1. Here, $\|x\|_0$ denotes the number of coefficients whose magnitude is greater than 10^{-6} .

Table 1: Numerical results for the ridge and sparse polynomial fits.

λ	Method	$\ \Phi x - y\ _2$	$\ x\ _2$	$\ x\ _0$
10^{-4}	Ridge	2.725978	87.03503	24
10^{-4}	Sparse	2.343260	1202.659	14
10^{-2}	Ridge	3.030231	59.61714	24
10^{-2}	Sparse	2.868227	65.06735	10
10^{-1}	Ridge	4.349349	57.68447	24
10^{-1}	Sparse	3.163258	60.29845	8
1	Ridge	14.21038	54.10571	24
1	Sparse	4.236113	59.02353	6
10	Ridge	79.92261	39.86945	24
10	Sparse	14.29139	56.67316	3
100	Ridge	225.4255	12.75882	24
100	Sparse	50.60797	56.08333	1

The ridge regularization shrinks the magnitude of the polynomial coefficients as λ increases, but all 24 coefficients remain nonzero. In contrast, the ℓ_1 -regularized solution becomes increasingly sparse as λ increases. The number of nonzero coefficients decreases from 14 to 1.

Thus, the ℓ_1 regularization produces significantly sparser polynomials, whereas the ℓ_2 regularization primarily reduces the magnitude of the coefficients.

Code for (2)

```

1     import numpy as np
2     import cvxpy as cp
3     import matplotlib.pyplot as plt
4
5     # Temperature measurements
6     t = np.arange(1, 25)
7     y = np.array([75, 77, 76, 73, 69, 68, 63, 59, 57, 55, 54, 52, 50, 50, 49, 49, 49,
8                   50, 54, 56, 59, 63, 67, 72], dtype=float)
9
10    # Normalize the sampling instants
11    s = (t - 12.5) / 11.5
12
13    # Construct the Vandermonde matrix
14    n = 24
15    Phi = np.vander(s, N=n, increasing=True)
16
17    # Interpolating polynomial
18    x_interp = np.linalg.solve(Phi, y)
19
20    # Values of lambda
21    lambdas = [1e-4, 1e-2, 1e-1, 1, 10, 100]
22
23    # Storage for coefficient vectors
24    ridge_coefficients = {}
25    sparse_coefficients = {}
26
27    # Solve ridge and sparse polynomial problems
28    for lam in lambdas:
29        x_ridge = cp.Variable(n)
30        ridge_problem = cp.Problem(cp.Minimize(0.5 * cp.sum_squares(Phi @ x_ridge - y) +
31        0.5 * lam * cp.sum_squares(x_ridge)))
32        ridge_problem.solve()
33
34        if x_ridge.value is not None:
35            ridge_coefficients[lam] = x_ridge.value.copy()
36
37        x_sparse = cp.Variable(n)
38        sparse_problem = cp.Problem(cp.Minimize(0.5 * cp.sum_squares(Phi @ x_sparse - y)
39        + lam * cp.norm1(x_sparse)))
40        sparse_problem.solve()
41
42        if x_sparse.value is not None:
43            sparse_coefficients[lam] = x_sparse.value.copy()
44
45    # Plotting points
46    s_plot = np.linspace(-1, 1, 500)
47    Phi_plot = np.vander(s_plot, N=n, increasing=True)
48
49    # Plot interpolating polynomial
50    y_interp = Phi_plot @ x_interp
51
52    plt.figure(figsize=(12, 7))
53    plt.scatter(s, y, label="Temperature measurements")
54    plt.plot(s_plot, y_interp, label="Interpolating polynomial")
55    plt.xlabel("Normalized sampling instant")
56    plt.ylabel("Temperature")
57    plt.title("Interpolating Polynomial")
58    plt.grid(True)
59    plt.legend()
60    plt.tight_layout()
61    plt.show()
62
63    # Plot ridge and sparse fits for each lambda
64    for lam in lambdas:
65        plt.figure(figsize=(12, 7))

```

```

63 plt.scatter(s, y, label="Temperature measurements")
64
65 if lam in ridge_coefficients:
66     y_ridge = Phi_plot @ ridge_coefficients[lam]
67     plt.plot(s_plot, y_ridge, label=f"Ridge, lambda = {lam}")
68
69 if lam in sparse_coefficients:
70     y_sparse = Phi_plot @ sparse_coefficients[lam]
71     plt.plot(s_plot, y_sparse, "--", color="orange", label=f"Sparse, lambda = {
72         lam}")
73
74 plt.xlabel("Normalized sampling instant")
75 plt.ylabel("Temperature")
76 plt.title(f"Polynomial Fits for lambda = {lam}")
77 plt.grid(True)
78 plt.legend()
79 plt.tight_layout()
80 plt.show()
81
82 # Plot coefficient vectors for each lambda
83 for lam in lambdas:
84     plt.figure(figsize=(12, 7))
85     x_axis = np.arange(n)
86
87     if lam in ridge_coefficients:
88         plt.plot(x_axis, ridge_coefficients[lam], label=f"Ridge, lambda = {lam}")
89
90     if lam in sparse_coefficients:
91         plt.plot(x_axis, sparse_coefficients[lam], "--", color="orange", label=f"
92             Sparse, lambda = {lam}")
93
94     plt.axhline(0, linewidth=0.8)
95     plt.xlabel("Coefficient index")
96     plt.ylabel("Coefficient value")
97     plt.title(f"Coefficient Vectors for lambda = {lam}")
98     plt.grid(True)
99     plt.legend()
100    plt.tight_layout()
101    plt.show()

```

Plots for (2)

The following plots show the interpolating polynomial, the ridge and sparse polynomial fits for each value of λ , and the corresponding coefficient vectors.

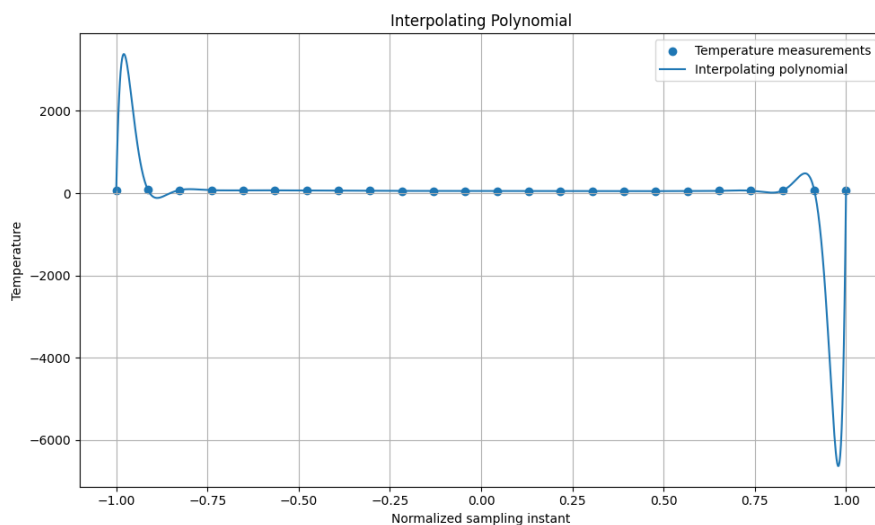


Figure 1: Interpolating polynomial

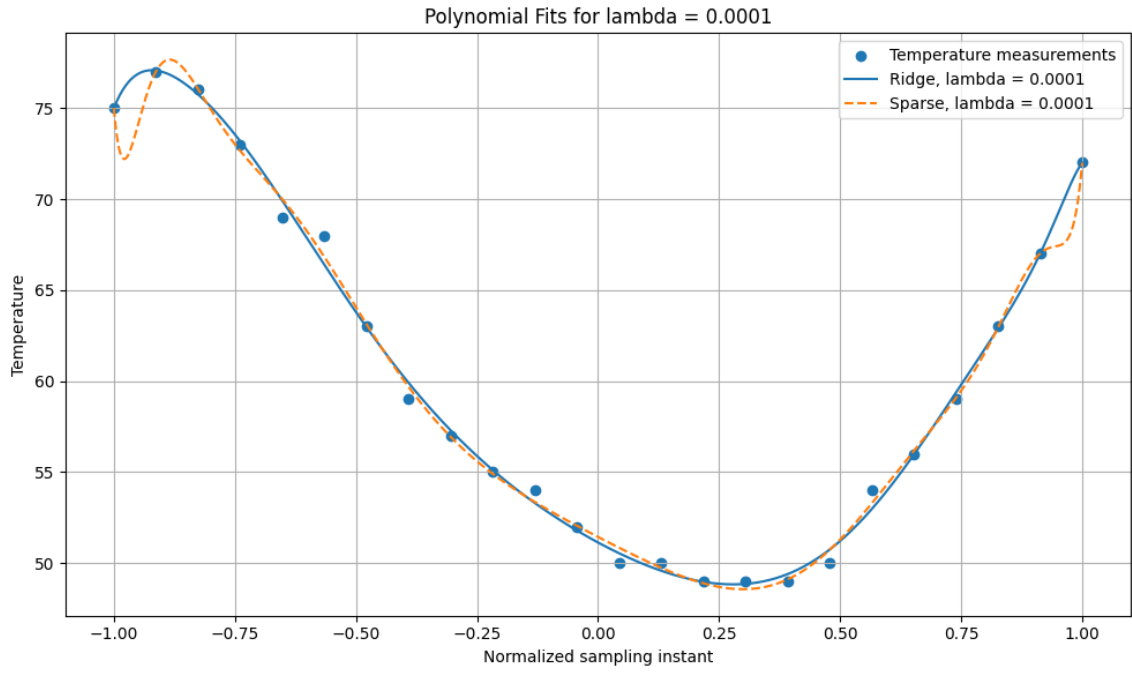


Figure 2: Polynomial fits for $\lambda = 10^{-4}$

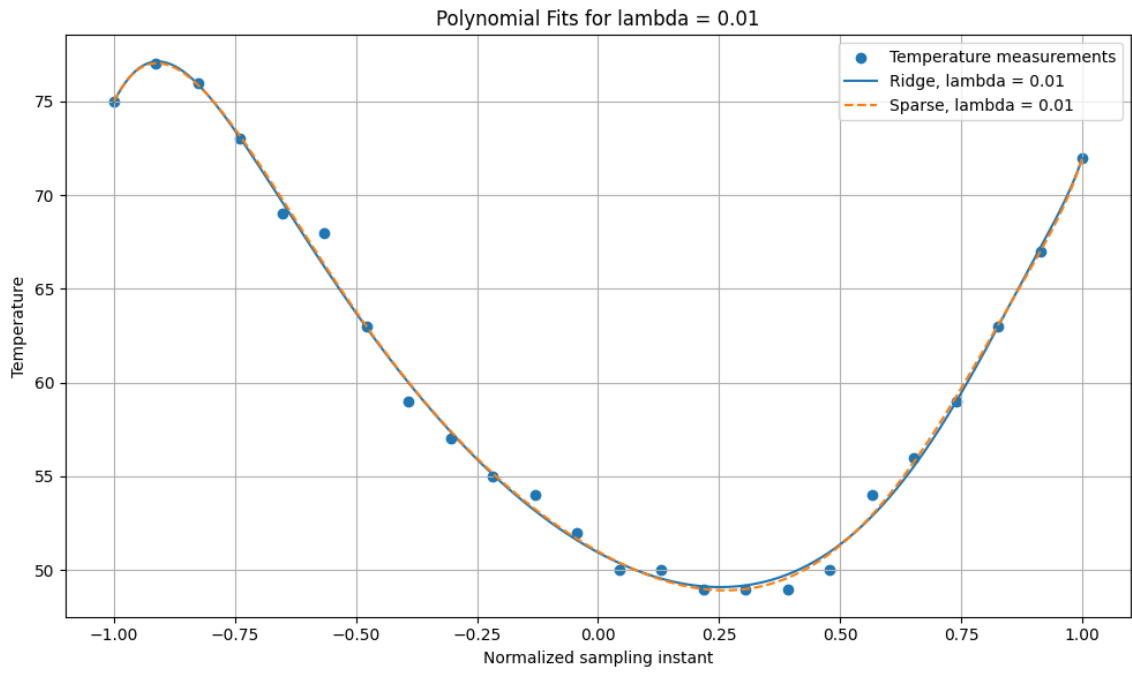


Figure 3: Polynomial fits for $\lambda = 10^{-2}$

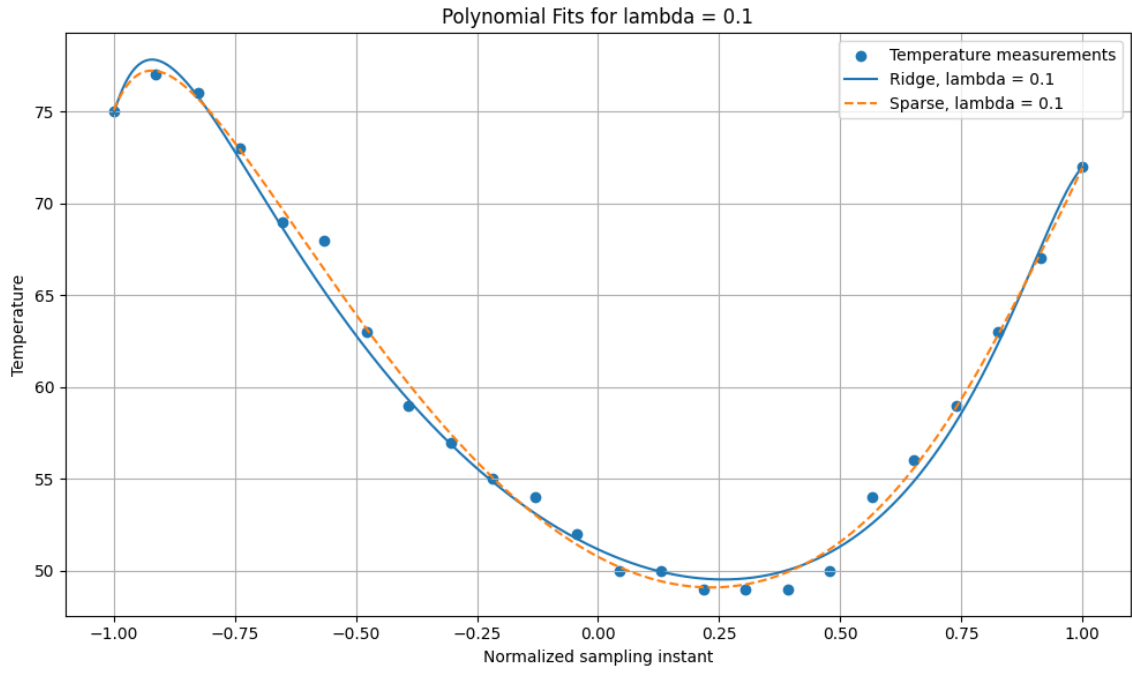


Figure 4: Polynomial fits for $\lambda = 10^{-1}$

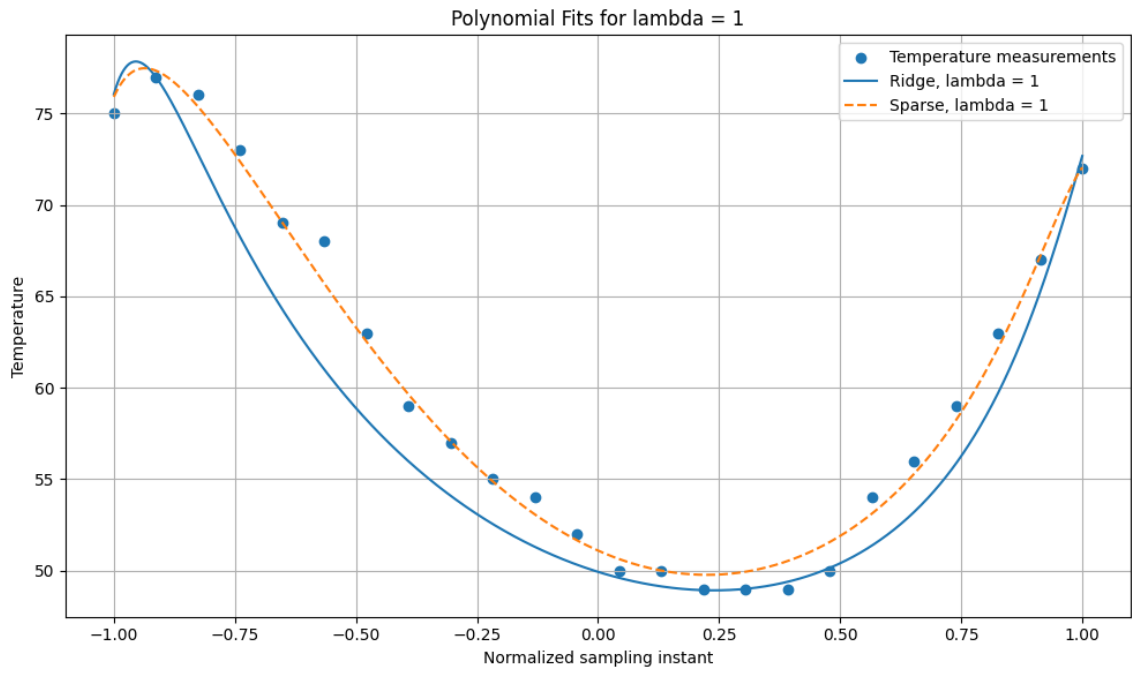


Figure 5: Polynomial fits for $\lambda = 1$

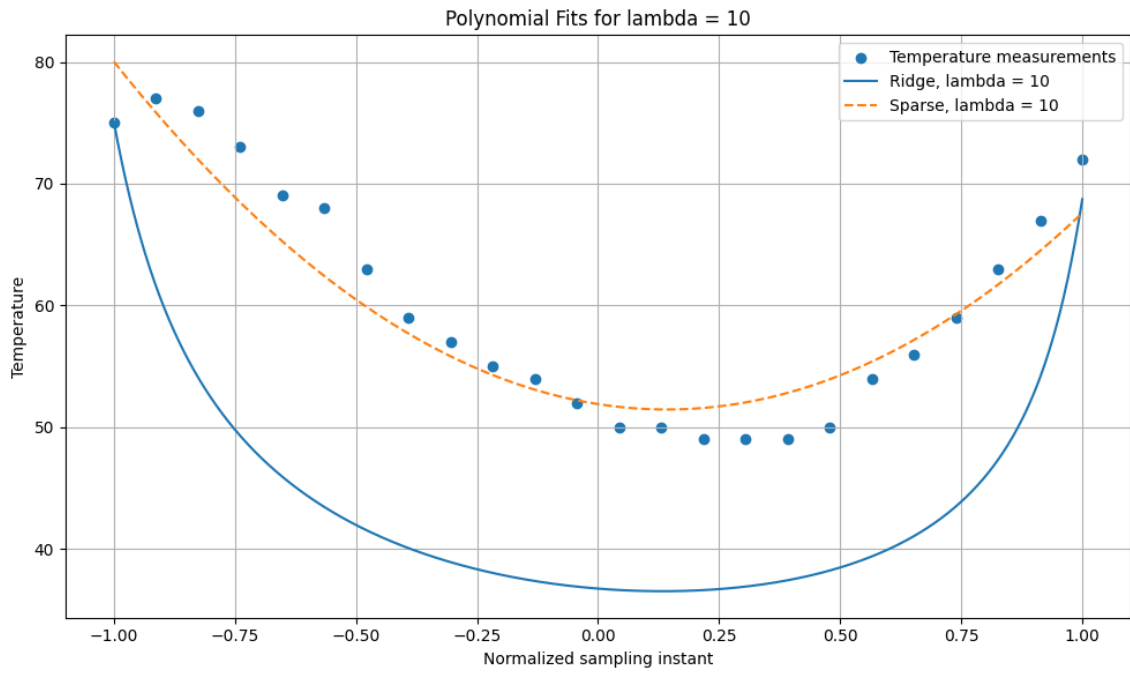


Figure 6: Polynomial fits for $\lambda = 10$

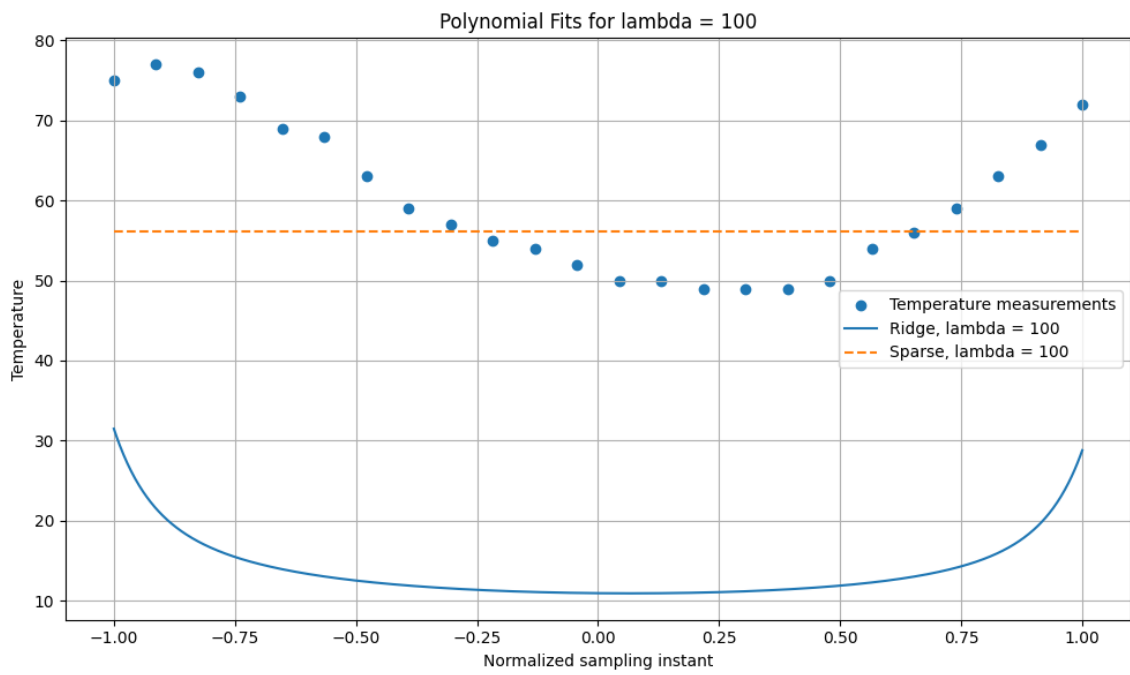


Figure 7: Polynomial fits for $\lambda = 100$

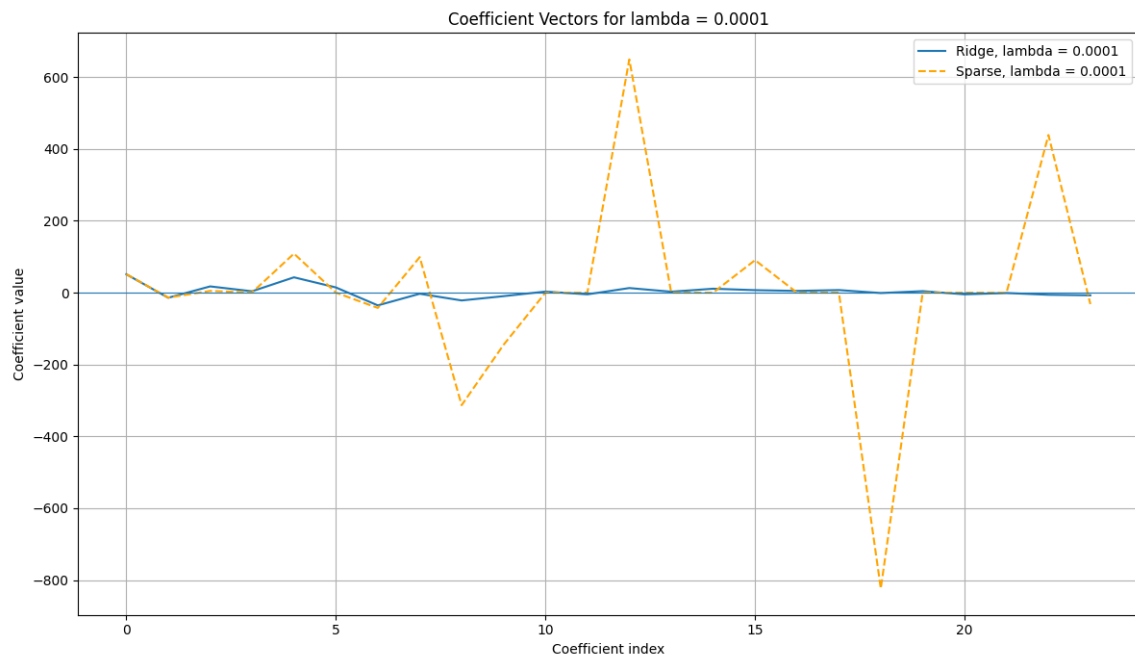


Figure 8: Coefficient vectors for $\lambda = 10^{-4}$

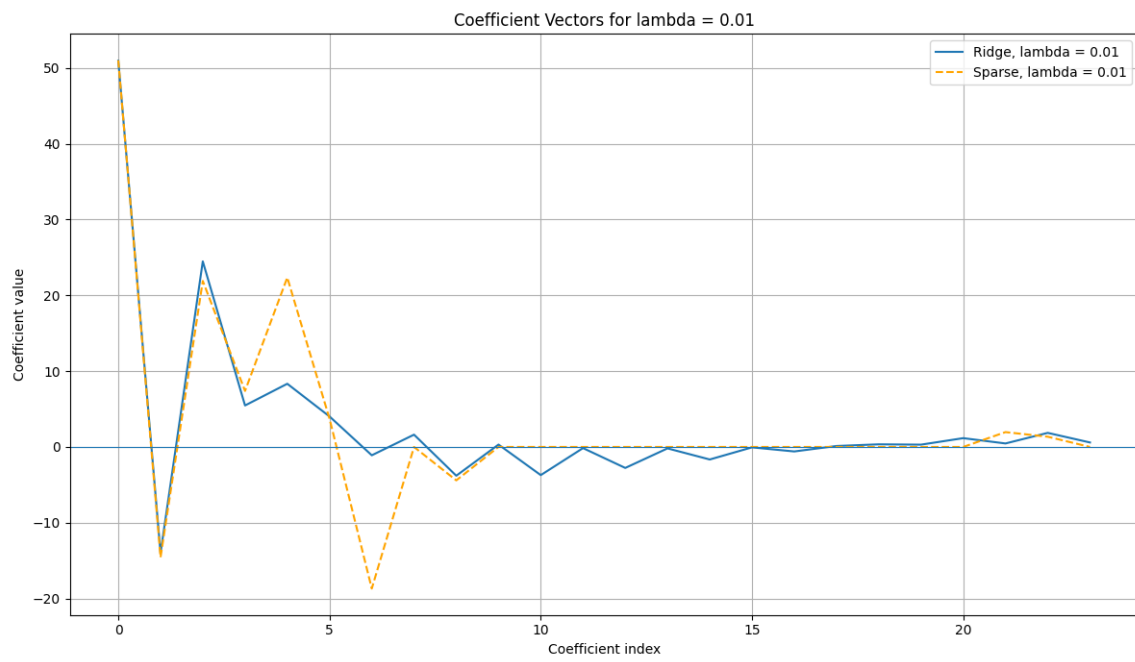


Figure 9: Coefficient vectors for $\lambda = 10^{-2}$

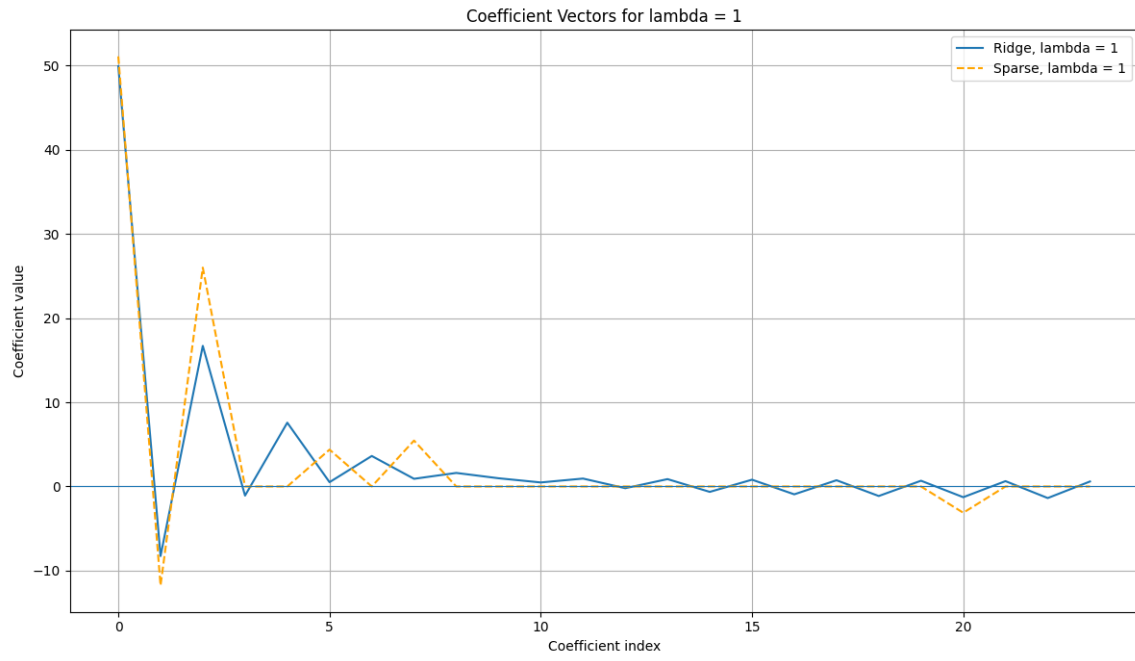


Figure 10: Coefficient vectors for $\lambda = 10^{-1}$

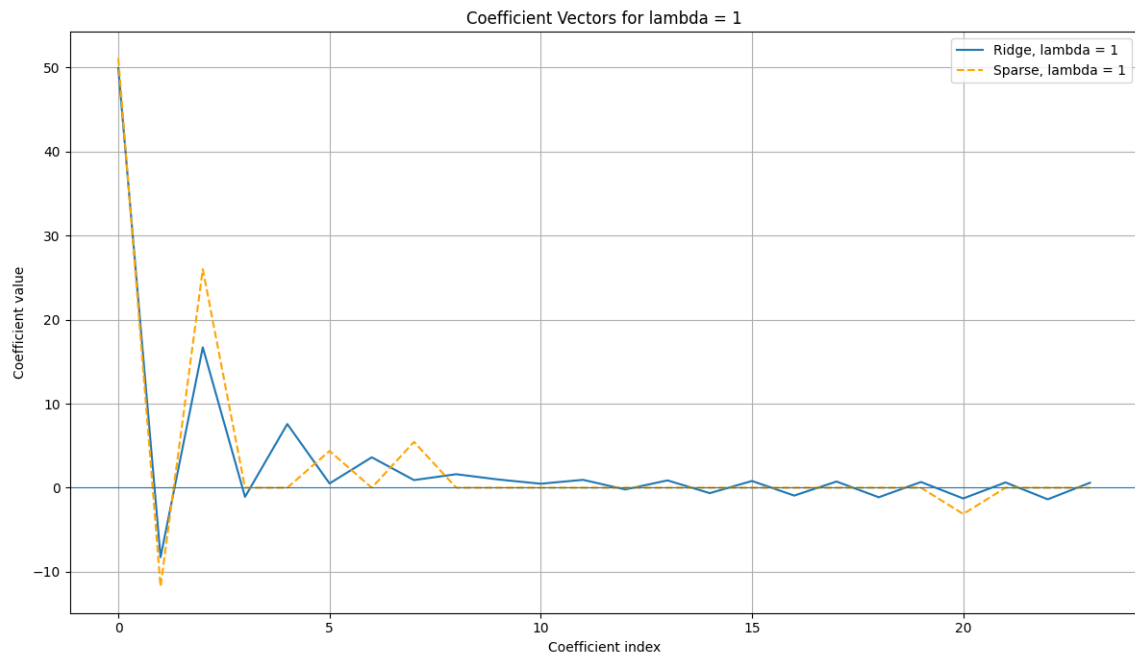


Figure 11: Coefficient vectors for $\lambda = 1$

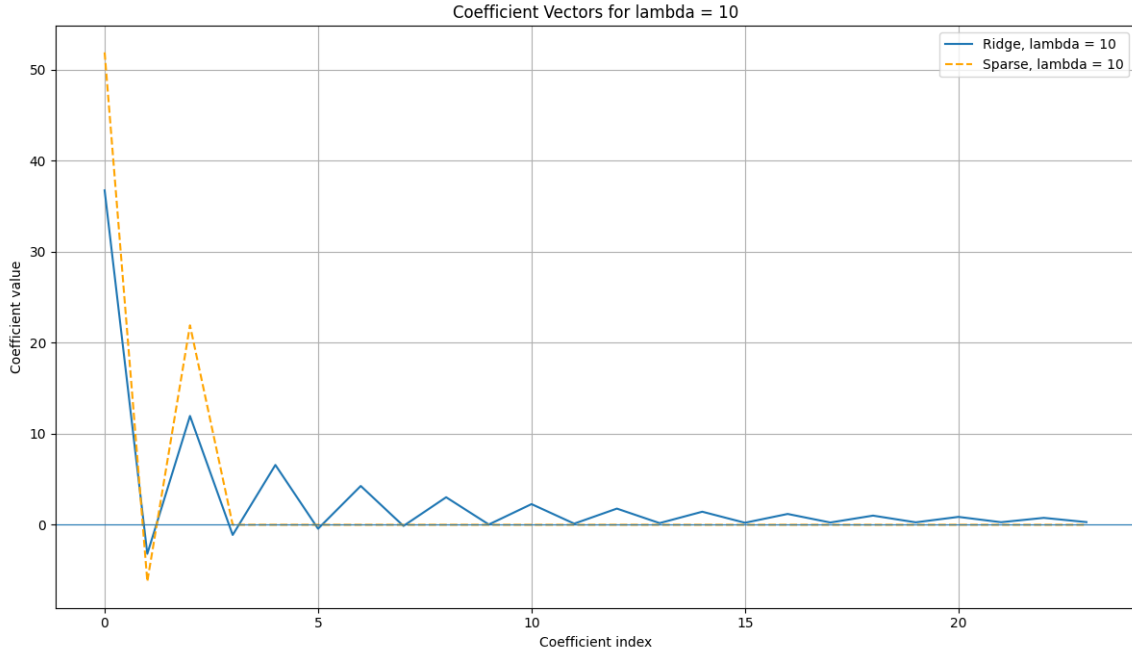


Figure 12: Coefficient vectors for $\lambda = 10$

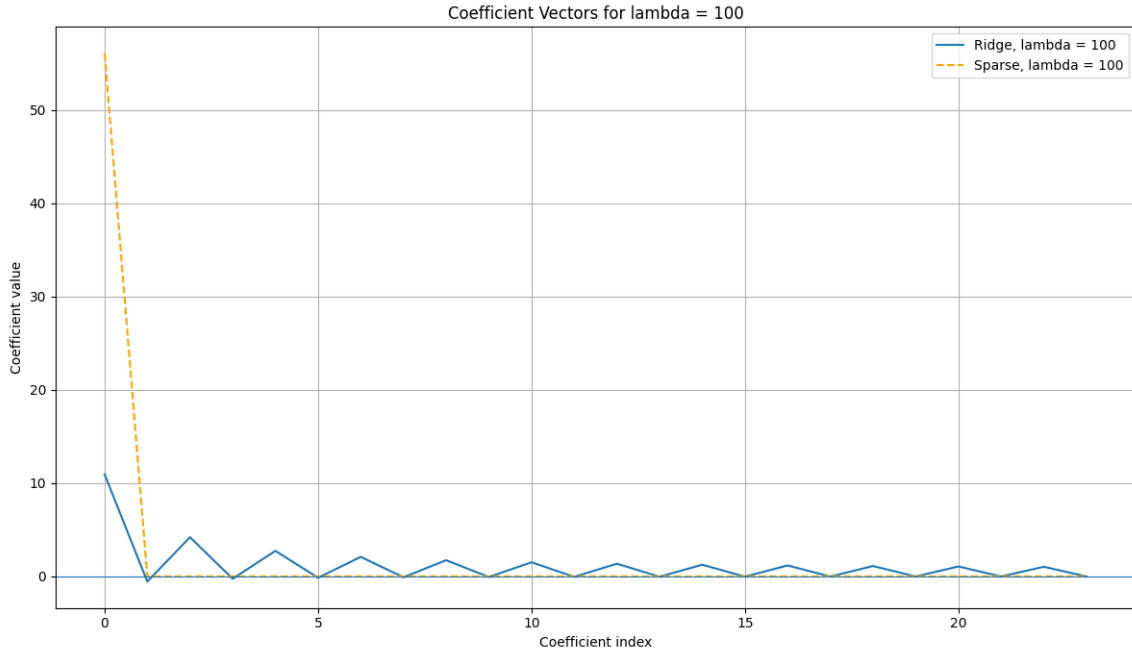


Figure 13: Coefficient vectors for $\lambda = 100$

Observations

The interpolating polynomial passes through all the temperature measurements, but oscillates near the boundaries. In particular, its values become extremely large near $x = -1$ and $x = 1$.

For the regularized solutions, the behavior changes with λ . For small values such as $\lambda = 10^{-4}$, both the Ridge and sparse solutions provide fits that are close to the measured data. However, the corresponding coefficient plots show that the sparse solution contains several very large coefficients, indicating that weak regularization does not sufficiently suppress the ill-conditioning of the polynomial basis. As λ increases to 10^{-2} and 10^{-1} , the Ridge and sparse fits become more stable and closely follow the measurements. The coefficient magnitudes are also reduced, while the sparse solution sets a larger number of coefficients exactly or approximately to zero.

At $\lambda = 1$, both regularization methods produce smooth fits that capture the overall quadratic shape of the temperature data. The sparse solution is slightly different from the Ridge solution but remains close to the measurements. Its coefficient vector is visibly more sparse

For larger regularization parameters, the effect of regularization becomes increasingly dominant. At $\lambda = 10$, the Ridge solution is noticeably flatter than the measured data, while the sparse solution provides a closer approximation to the overall trend. At $\lambda = 100$, the Ridge polynomial is strongly shrunk toward smaller coefficient values, resulting in a substantial underfit. The sparse solution reduces almost all coefficients to zero except for the leading coefficient, producing an approximately constant fit. Thus, excessively large values of λ cause both methods to lose important information from the data.

From the coefficient plots, we can see that Ridge regularization reduces the magnitude of the coefficients but generally keeps most coefficients nonzero. In contrast, the sparse regularization produces coefficient vectors with many zero coefficients, giving a polynomial representation. Overall, moderate regularization, particularly around $\lambda = 10^{-2}$ to $\lambda = 1$, provides a good balance between fitting the measurements, controlling the large oscillations of the interpolating polynomial, and maintaining a relatively simple coefficient representation.

Code for (3)

```

1  import numpy as np
2  import cvxpy as cp
3  import matplotlib.pyplot as plt
4  import os
5
6  # Temperature measurements
7  t = np.arange(1, 25)
8  y = np.array([75, 77, 76, 73, 69, 68, 63, 59, 57, 55, 54, 52,
9               50, 50, 49, 49, 49, 50, 54, 56, 59, 63, 67, 72])
10
11 # Normalize sampling instants
12 s = (t - 12.5) / 11.5
13
14 # Vandermonde matrix for a polynomial of order 23
15 Phi = np.vander(s, 24, increasing=True)
16
17 # Regularization parameters
18 lambdas = [1e-4, 1e-2, 1e-1, 1, 10, 100]
19
20 ridge_errors = []
21 sparse_errors = []
22 ridge_nnz = []
23 sparse_nnz = []
24
25 for lam in lambdas:
26     # Ridge regression
27     x = cp.Variable(24)
28     problem = cp.Problem(
29         cp.Minimize(0.5 * cp.sum_squares(Phi @ x - y) +
30                     0.5 * lam * cp.sum_squares(x))
31     )
32     problem.solve()
33     ridge_x = x.value
34
35     # Sparse regression
36     x = cp.Variable(24)
37     problem = cp.Problem(
38         cp.Minimize(0.5 * cp.sum_squares(Phi @ x - y) +
39                     lam * cp.norm1(x))
40     )
41     problem.solve()
42     sparse_x = x.value
43
44     # Relative fitting errors
45     ridge_error = np.linalg.norm(Phi @ ridge_x - y) / np.linalg.norm(y)
46     sparse_error = np.linalg.norm(Phi @ sparse_x - y) / np.linalg.norm(y)
47
48     # Number of nonzero coefficients

```

```

49     ridge_count = np.count_nonzero(np.abs(ridge_x) > 1e-6)
50     sparse_count = np.count_nonzero(np.abs(sparse_x) > 1e-6)
51
52     ridge_errors.append(ridge_error)
53     sparse_errors.append(sparse_error)
54     ridge_nnz.append(ridge_count)
55     sparse_nnz.append(sparse_count)
56
57     print(f"lambda = {lam}")
58     print(f"Ridge : E(x) = {ridge_error:.6f}, ||x||_0 = {ridge_count}")
59     print(f"Sparse : E(x) = {sparse_error:.6f}, ||x||_0 = {sparse_count}")
60     print()
61
62     # Create output directory
63     os.makedirs("images", exist_ok=True)
64
65     # Plot relative fitting error
66     plt.figure(figsize=(8, 5))
67     plt.semilogx(lambdas, ridge_errors, 'o-', label='Ridge')
68     plt.semilogx(lambdas, sparse_errors, 'o--', label='Sparse')
69     plt.xlabel(r'$\lambda$')
70     plt.ylabel(r'Relative fitting error $E(x)$')
71     plt.title('Relative Fitting Error vs. Regularization')
72     plt.grid(True)
73     plt.legend()
74     plt.tight_layout()
75     plt.savefig("images/Q5_3_fitting_error.png", dpi=300)
76     plt.show()
77
78     # Plot sparsity
79     plt.figure(figsize=(8, 5))
80     plt.semilogx(lambdas, ridge_nnz, 'o-', label='Ridge')
81     plt.semilogx(lambdas, sparse_nnz, 'o--', label='Sparse')
82     plt.xlabel(r'$\lambda$')
83     plt.ylabel(r'Number of nonzero coefficients')
84     plt.title('Sparsity vs. Regularization')
85     plt.grid(True)
86     plt.legend()
87     plt.tight_layout()
88     plt.savefig("images/Q5_3_sparsity.png", dpi=300)
89     plt.show()

```

Plots for (3)

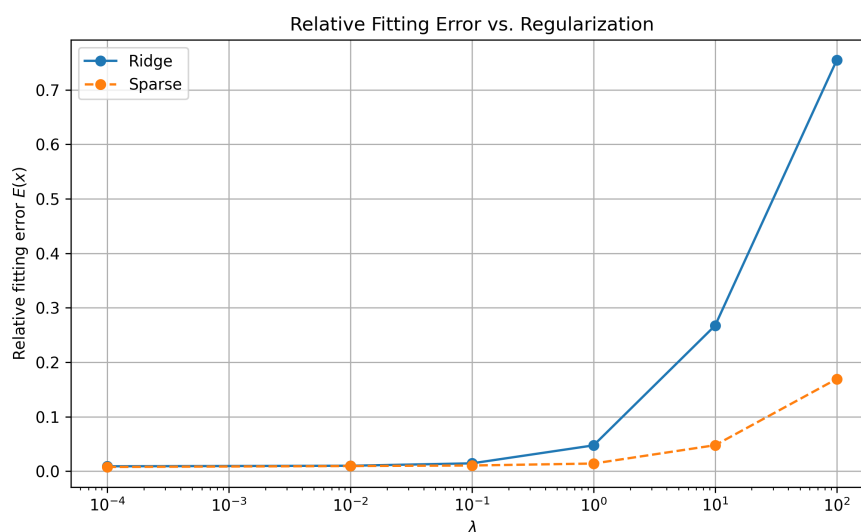


Figure 14: Relative fitting error as a function of the regularization parameter λ .

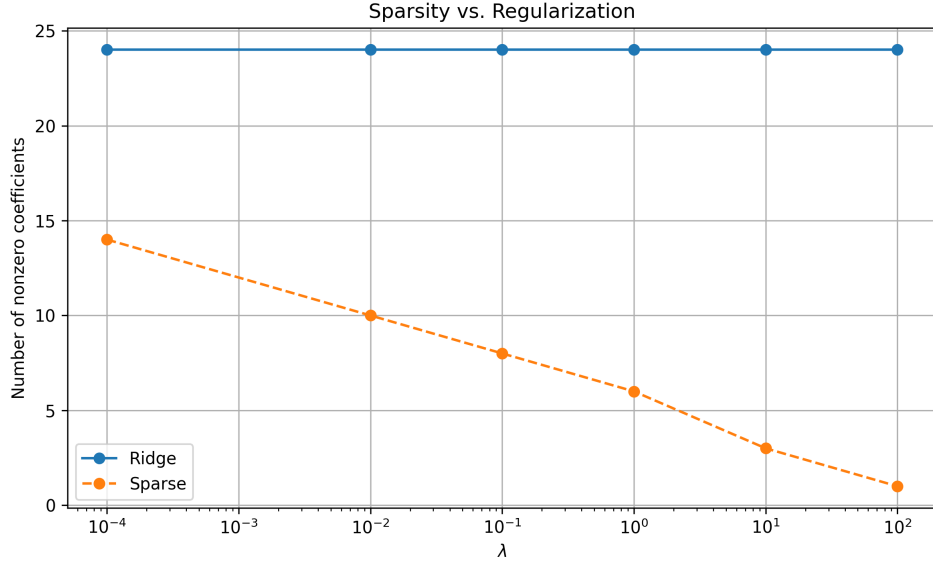


Figure 15: Number of nonzero coefficients as a function of the regularization parameter λ .

Observations

The relative fitting error for both Ridge and sparse regularization is small for low values of λ . As λ increases, the fitting error increases because stronger regularization increasingly restricts the polynomial coefficients. The increase is much more for Ridge regularization. At $\lambda = 100$, the Ridge fitting error reaches approximately 0.75, whereas the sparse model has an error of approximately 0.17.

The sparsity plot shows a clear difference between the two regularization methods. Ridge regularization has 24 nonzero coefficients for all values of λ . This is because the L_2 penalty shrinks the coefficients toward zero but does not generally make them exactly zero.

In contrast, the sparse L_1 regularization progressively reduces the number of nonzero coefficients as λ increases. The number of nonzero coefficients decreases from 14 at $\lambda = 10^{-4}$ to only 1 at $\lambda = 100$. Thus, the L_1 penalty promotes sparsity much more effectively than the L_2 penalty.

Overall, the sparse formulation provides a better trade-off between fitting accuracy and model complexity for the values of λ considered. In particular, at $\lambda = 1$, the sparse model has a relative fitting error of only 0.014178 while using just 6 nonzero coefficients, compared with a Ridge error of 0.047562 with all 24 coefficients remaining nonzero.

Problem 6

Exercise 4.14: Assume that f is a proper, closed, and convex function, $\gamma > 0$, and $v \in \mathbb{R}^n$. Prove that the function

$$g(x) \triangleq f(x) + \frac{1}{2\gamma} \|x - v\|_2^2$$

is proper, closed, and strongly convex with modulus

$$\beta = \frac{1}{\gamma}.$$

Solution

1. Proving that g is proper

Since f is proper, it is not equal to $+\infty$, and it never takes the value $-\infty$. Therefore, there exists some $x_0 \in \mathbb{R}^n$ such that

$$f(x_0) < +\infty.$$

Since $\gamma > 0$ and $v \in \mathbb{R}^n$, the quadratic term

$$\frac{1}{2\gamma}\|x_0 - v\|_2^2$$

is finite. Hence,

$$g(x_0) = f(x_0) + \frac{1}{2\gamma}\|x_0 - v\|_2^2 < +\infty.$$

Also, since $f(x)$ never takes the value $-\infty$ and the quadratic term is finite for every $x \in \mathbb{R}^n$, we have

$$g(x) > -\infty$$

for every x .

Therefore, g is proper.

2. Proving that g is closed

Define

$$h(x) = \frac{1}{2\gamma}\|x - v\|_2^2.$$

The function h is a quadratic function of x . Since $\gamma > 0$, it is continuous on $\mathbb{R}^n$. We are given that f is closed. Since h is finite and continuous everywhere, the sum

$$g(x) = f(x) + h(x)$$

is also closed.

Therefore,

$$\boxed{g \text{ is closed.}}$$

3. Proving that g is strongly convex

Let $x, y \in \mathbb{R}^n$ and let $t \in [0, 1]$. We consider

$$h(tx + (1-t)y).$$

Substituting the definition of h , we get

$$h(tx + (1-t)y) = \frac{1}{2\gamma}\|tx + (1-t)y - v\|_2^2.$$

We can rewrite the expression inside the norm as

$$tx + (1-t)y - v = t(x - v) + (1-t)(y - v).$$

Therefore,

$$h(tx + (1-t)y) = \frac{1}{2\gamma}\|t(x - v) + (1-t)(y - v)\|_2^2.$$

Using the identity

$$\|ta + (1-t)b\|_2^2 = t\|a\|_2^2 + (1-t)\|b\|_2^2 - t(1-t)\|a - b\|_2^2,$$

with

$$a = x - v, \quad b = y - v,$$

we obtain

$$\|t(x - v) + (1 - t)(y - v)\|_2^2 = t\|x - v\|_2^2 + (1 - t)\|y - v\|_2^2 - t(1 - t)\|(x - v) - (y - v)\|_2^2.$$

Since

$$(x - v) - (y - v) = x - y,$$

this becomes

$$\|t(x - v) + (1 - t)(y - v)\|_2^2 = t\|x - v\|_2^2 + (1 - t)\|y - v\|_2^2 - t(1 - t)\|x - y\|_2^2.$$

Hence,

$$h(tx + (1 - t)y) = \frac{t}{2\gamma}\|x - v\|_2^2 + \frac{1 - t}{2\gamma}\|y - v\|_2^2 - \frac{t(1 - t)}{2\gamma}\|x - y\|_2^2.$$

Recognizing the first two terms as $th(x)$ and $(1 - t)h(y)$, we get

$$h(tx + (1 - t)y) = th(x) + (1 - t)h(y) - \frac{t(1 - t)}{2\gamma}\|x - y\|_2^2.$$

Thus, the quadratic function h is strongly convex with modulus

$$\frac{1}{\gamma}.$$

Now, Since f is convex, for every $x, y \in \mathbb{R}^n$ and $t \in [0, 1]$,

$$f(tx + (1 - t)y) \leq tf(x) + (1 - t)f(y).$$

Now, by definition,

$$g(tx + (1 - t)y) = f(tx + (1 - t)y) + h(tx + (1 - t)y).$$

Using the convexity inequality for f and the strong convexity relation for h , we obtain

$$g(tx + (1 - t)y) \leq tf(x) + (1 - t)f(y) + th(x) + (1 - t)h(y) - \frac{t(1 - t)}{2\gamma}\|x - y\|_2^2.$$

Grouping the terms involving f and h , we get

$$g(tx + (1 - t)y) \leq t(f(x) + h(x)) + (1 - t)(f(y) + h(y)) - \frac{t(1 - t)}{2\gamma}\|x - y\|_2^2.$$

Since

$$g(x) = f(x) + h(x)$$

and

$$g(y) = f(y) + h(y),$$

we have

$$g(tx + (1-t)y) \leq tg(x) + (1-t)g(y) - \frac{t(1-t)}{2\gamma} \|x - y\|_2^2.$$

For the convexity modulus, We can compare the inequality above with

$$g(tx + (1-t)y) \leq tg(x) + (1-t)g(y) - \frac{\beta}{2} t(1-t) \|x - y\|_2^2$$

From that, we get that

$$\frac{\beta}{2} = \frac{1}{2\gamma}.$$

Therefore,

$$\boxed{\beta = \frac{1}{\gamma}}.$$

Hence, g is strongly convex with modulus $1/\gamma$.

□

Exercise 4.22. Show the cost function $\|x\|_1$ in

$$\min_{x \in \mathbb{R}^n} \|x\|_1 \quad \text{subject to} \quad \Phi x = y,$$

is a proper, closed, and convex function. Also, show the set $\mathcal{C}$ defined in

$$\mathcal{C} \triangleq \{x \in \mathbb{R}^n : \Phi x = y\}.$$

is a non-empty, closed, and convex set in $\mathbb{R}^n$.

Solution.

1. The function $\|x\|_1$ is proper

The ℓ^1 norm is defined as

$$\|x\|_1 = \sum_{i=1}^n |x_i|.$$

For every $x \in \mathbb{R}^n$, each $|x_i|$ is finite and non-negative. Therefore,

$$0 \leq \|x\|_1 < +\infty$$

for every $x \in \mathbb{R}^n$.

In particular,

$$\|0\|_1 = 0 < +\infty.$$

Thus, $\|x\|_1$ is never equal to $-\infty$ and is not equal to $+\infty$. Hence, $\|x\|_1$ is a proper function.

Therefore,

$$\boxed{\|x\|_1 \text{ is proper.}}$$

2. The function $\|x\|_1$ is closed

The function $|x_i|$ is continuous for every i . Therefore, the sum

$$\|x\|_1 = \sum_{i=1}^n |x_i|$$

is also continuous on $\mathbb{R}^n$.

Since every finite-valued continuous function is closed. Hence,

$$\boxed{\|x\|_1 \text{ is closed.}}$$

3. The function $\|x\|_1$ is convex

Let $x, y \in \mathbb{R}^n$ and let $t \in [0, 1]$.

We have

$$\|tx + (1-t)y\|_1 = \sum_{i=1}^n |tx_i + (1-t)y_i|.$$

Using the triangle inequality,

$$|tx_i + (1-t)y_i| \leq |tx_i| + |(1-t)y_i|.$$

Since $t \geq 0$ and $1-t \geq 0$,

$$|tx_i + (1-t)y_i| \leq t|x_i| + (1-t)|y_i|.$$

Summing over all i , we obtain

$$\begin{aligned} \|tx + (1-t)y\|_1 &\leq \sum_{i=1}^n (t|x_i| + (1-t)|y_i|) \\ &= t \sum_{i=1}^n |x_i| + (1-t) \sum_{i=1}^n |y_i| \\ &= t\|x\|_1 + (1-t)\|y\|_1. \end{aligned}$$

Hence,

$$\boxed{\|x\|_1 \text{ is convex.}}$$

Thus, the cost function $\|x\|_1$ is proper, closed, and convex.

4. The set $\mathcal{C}$ is non-empty

Recall that

$$\mathcal{C} = \{x \in \mathbb{R}^n : \Phi x = y\}.$$

We are given that

$$\Phi \in \mathbb{R}^{m \times n}$$

has full row rank, so

$$\text{rank}(\Phi) = m.$$

Since Φ maps $\mathbb{R}^n$ into $\mathbb{R}^m$, its range has dimension

$$\dim(\text{Range}(\Phi)) = \text{rank}(\Phi) = m.$$

But $\mathbb{R}^m$ itself has dimension m . Therefore,

$$\text{Range}(\Phi) = \mathbb{R}^m.$$

Hence, for the given $y \in \mathbb{R}^m$, there exists some $x \in \mathbb{R}^n$ such that

$$\Phi x = y.$$

Therefore,

$$x \in \mathcal{C},$$

which means

$$\boxed{\mathcal{C} \neq \emptyset.}$$

5. The set $\mathcal{C}$ is closed

Consider the linear mapping

$$\Phi : \mathbb{R}^n \rightarrow \mathbb{R}^m.$$

Every linear mapping between finite-dimensional vector spaces is continuous.

The set $\{y\}$ is a closed subset of $\mathbb{R}^m$. The set $\mathcal{C}$ can be written as the preimage of this closed set:

$$\mathcal{C} = \Phi^{-1}(\{y\}).$$

Since Φ is continuous and $\{y\}$ is closed, its preimage is closed. Therefore,

$$\boxed{\mathcal{C} \text{ is closed.}}$$

6. The set $\mathcal{C}$ is convex

Let $x_1, x_2 \in \mathcal{C}$ and let $t \in [0, 1]$.

Since $x_1, x_2 \in \mathcal{C}$, they satisfy the constraint

$$\Phi x_1 = y$$

and

$$\Phi x_2 = y.$$

Consider the convex combination

$$x = tx_1 + (1 - t)x_2.$$

Using the linearity of Φ ,

$$\begin{aligned}\Phi x &= \Phi(tx_1 + (1-t)x_2) \\ &= t\Phi x_1 + (1-t)\Phi x_2.\end{aligned}$$

Substituting $\Phi x_1 = y$ and $\Phi x_2 = y$, we obtain

$$\begin{aligned}\Phi x &= ty + (1-t)y \\ &= (t + 1 - t)y \\ &= y.\end{aligned}$$

Therefore,

$$x = tx_1 + (1-t)x_2 \in \mathcal{C}.$$

Since this holds for every $x_1, x_2 \in \mathcal{C}$ and every $t \in [0, 1]$, the set $\mathcal{C}$ is convex.

Thus,

$$\boxed{\mathcal{C} \text{ is convex.}}$$

□

Exercise 4.23. Suppose that two functions, f and g , are proper, closed, and convex. Suppose also that $\text{dom}(f) \cap \text{dom}(g) \neq \emptyset$. Prove that $f + g$ is also a proper, closed, and convex function.

Solution:

1. Proving that $f + g$ is proper

Since f and g are proper, neither function takes the value $-\infty$. Therefore, their sum also cannot take the value $-\infty$. Thus, $f + g$ is not identically equal to $+\infty$ and never takes the value $-\infty$. Hence,

$$\boxed{f + g \text{ is proper.}}$$

2. Proving that $f + g$ is convex

Let $x, y \in \mathbb{R}^n$ and let $t \in [0, 1]$.

Since f and g are convex,

$$f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$$

and

$$g(tx + (1-t)y) \leq tg(x) + (1-t)g(y).$$

Adding the two inequalities gives

$$\begin{aligned}(f + g)(tx + (1-t)y) &= f(tx + (1-t)y) + g(tx + (1-t)y) \\ &\leq tf(x) + (1-t)f(y) + tg(x) + (1-t)g(y) \\ &= t(f(x) + g(x)) + (1-t)(f(y) + g(y)) \\ &= t(f + g)(x) + (1-t)(f + g)(y).\end{aligned}$$

Hence,

$$\boxed{f + g \text{ is convex.}}$$

3. Proving that $f + g$ is closed

The effective domain of $f + g$ is

$$\text{dom}(f + g) = \text{dom}(f) \cap \text{dom}(g).$$

Since f and g are closed convex functions, their epigraphs are closed:

$$\text{epi}(f) = \{(x, \alpha) : f(x) \leq \alpha\}$$

and

$$\text{epi}(g) = \{(x, \alpha) : g(x) \leq \alpha\}.$$

For the sum, we have

$$\text{epi}(f + g) = \{(x, \alpha) : f(x) + g(x) \leq \alpha\}.$$

To show that this set is closed, consider a sequence

$$(x_k, \alpha_k) \rightarrow (x, \alpha)$$

such that

$$f(x_k) + g(x_k) \leq \alpha_k$$

for every k .

Because f and g are closed, they are lower semicontinuous. Therefore,

$$f(x) \leq \liminf_{k \rightarrow \infty} f(x_k)$$

and

$$g(x) \leq \liminf_{k \rightarrow \infty} g(x_k).$$

Adding these inequalities gives

$$f(x) + g(x) \leq \liminf_{k \rightarrow \infty} (f(x_k) + g(x_k)).$$

Since

$$f(x_k) + g(x_k) \leq \alpha_k$$

and $\alpha_k \rightarrow \alpha$, we have

$$\liminf_{k \rightarrow \infty} (f(x_k) + g(x_k)) \leq \alpha.$$

Hence,

$$f(x) + g(x) \leq \alpha.$$

Therefore,

$$(x, \alpha) \in \text{epi}(f + g).$$

Thus, $\text{epi}(f + g)$ is closed, and consequently $f + g$ is closed.

Therefore,

$$\boxed{f + g \text{ is closed.}}$$

Problem 7

Exercise 4.19. Show that the minimizer x^* of the function

$$f(x) \triangleq |x| + \frac{1}{2\gamma}(x - v)^2$$

is given by

$$x^* = S_\gamma(v) \triangleq \begin{cases} v - \gamma, & \text{if } v \geq \gamma, \\ 0, & \text{if } -\gamma < v < \gamma, \\ v + \gamma, & \text{if } v \leq -\gamma. \end{cases}$$

Solution:

Case 1: $x \geq 0$

For $x \geq 0$, we have $|x| = x$. Therefore,

$$f(x) = x + \frac{1}{2\gamma}(x - v)^2.$$

Differentiating with respect to x ,

$$f'(x) = 1 + \frac{1}{\gamma}(x - v).$$

Setting the derivative equal to zero gives

$$1 + \frac{1}{\gamma}(x - v) = 0.$$

Hence,

$$x - v = -\gamma,$$

and therefore

$$x = v - \gamma.$$

Since this case requires $x \geq 0$, we must have

$$v - \gamma \geq 0,$$

which gives

$$v \geq \gamma.$$

Thus, when $v \geq \gamma$,

$$x^* = v - \gamma.$$

Case 2: $x < 0$

For $x < 0$, we have $|x| = -x$. Therefore,

$$f(x) = -x + \frac{1}{2\gamma}(x - v)^2.$$

Differentiating with respect to x ,

$$f'(x) = -1 + \frac{1}{\gamma}(x - v).$$

Setting the derivative equal to zero gives

$$-1 + \frac{1}{\gamma}(x - v) = 0.$$

Hence,

$$x - v = \gamma,$$

and therefore

$$x = v + \gamma.$$

Since this case requires $x < 0$, we must have

$$v + \gamma < 0,$$

which gives

$$v < -\gamma.$$

Thus, when $v < -\gamma$,

$$x^* = v + \gamma.$$

Case 3: $-\gamma < v < \gamma$

In this case, neither of the stationary points obtained above lies in its corresponding interval.

For $x \geq 0$, the stationary point is

$$x = v - \gamma < 0,$$

so it is not feasible in this interval. For $x < 0$, the stationary point is

$$x = v + \gamma > 0,$$

so it is not feasible in this interval. Therefore, the minimum must occur at the boundary between the two intervals, namely

$$x = 0.$$

Thus, when

$$-\gamma < v < \gamma,$$

we have

$$x^* = 0.$$

Combining all three cases, we obtain

$$x^* = \begin{cases} v - \gamma, & \text{if } v \geq \gamma, \\ 0, & \text{if } -\gamma < v < \gamma, \\ v + \gamma, & \text{if } v \leq -\gamma. \end{cases}$$

Therefore,

$$\boxed{x^* = S_\gamma(v)}.$$

□

Exercise 4.21. Compute the proximal operator of the ℓ^0 norm, and show that it is the hard-thresholding operator.

$$H_\lambda(v) \triangleq \begin{cases} v, & \text{if } |v| \geq \lambda, \\ 0, & \text{if } |v| < \lambda. \end{cases}$$

Solution

Let $x, v \in \mathbb{R}^n$. The ℓ^0 norm counts the number of nonzero entries of x , so

$$\|x\|_0 = \sum_{i=1}^n \mathbb{I}(x_i \neq 0).$$

Also,

$$\|x - v\|_2^2 = \sum_{i=1}^n (x_i - v_i)^2.$$

The minimization problem can be written as

$$\text{prox}_{\gamma f}(v) = \arg \min_x \sum_{i=1}^n \left[\mathbb{I}(x_i \neq 0) + \frac{1}{2\gamma} (x_i - v_i)^2 \right].$$

Since the objective is separable in the components of x , we can minimize each component independently. For a particular component, we need to solve

$$\min_{x_i} \left\{ \mathbb{I}(x_i \neq 0) + \frac{1}{2\gamma} (x_i - v_i)^2 \right\}.$$

Case 1: $x_i = 0$

If $x_i = 0$, then

$$\mathbb{I}(x_i \neq 0) = 0.$$

Hence, the objective becomes

$$\frac{1}{2\gamma} (0 - v_i)^2 = \frac{v_i^2}{2\gamma}.$$

Thus, the cost of choosing $x_i = 0$ is

$$C_0 = \frac{v_i^2}{2\gamma}.$$

Case 2: $x_i \neq 0$

If $x_i \neq 0$, then

$$\mathbb{I}(x_i \neq 0) = 1.$$

Therefore, the objective becomes

$$1 + \frac{1}{2\gamma} (x_i - v_i)^2.$$

The quadratic term is minimized when

$$x_i = v_i.$$

Thus, when $x_i \neq 0$, the minimum possible cost is

$$C_1 = 1.$$

Therefore, we compare

$$\frac{v_i^2}{2\gamma} \quad \text{and} \quad 1.$$

We choose $x_i = v_i$ whenever

$$1 \leq \frac{v_i^2}{2\gamma}.$$

Multiplying by 2γ gives

$$v_i^2 \geq 2\gamma.$$

Equivalently,

$$|v_i| \geq \sqrt{2\gamma}.$$

On the other hand, if

$$|v_i| < \sqrt{2\gamma},$$

then

$$\frac{v_i^2}{2\gamma} < 1,$$

so it is cheaper to choose

$$x_i = 0.$$

Hence,

$$x_i^* = \begin{cases} v_i, & |v_i| \geq \sqrt{2\gamma}, \\ 0, & |v_i| < \sqrt{2\gamma}. \end{cases}$$

Defining

$$\lambda = \sqrt{2\gamma},$$

we obtain

$$x_i^* = \begin{cases} v_i, & |v_i| \geq \lambda, \\ 0, & |v_i| < \lambda. \end{cases}$$

Applying this component-wise gives

$$\text{prox}_{\gamma\|\cdot\|_0}(v) = H_\lambda(v),$$

where

$$H_\lambda(v) \triangleq \begin{cases} v, & \text{if } |v| \geq \lambda, \\ 0, & \text{if } |v| < \lambda, \end{cases}$$

with

$$\lambda = \sqrt{2\gamma}.$$

□

Exercise 4.24. Show that the projection operator Π_C for C defined by

$$C \triangleq \{x \in \mathbb{R}^n : \Phi x = y\}$$

is given by

$$\Pi_C(v) = v + \Phi^\top (\Phi \Phi^\top)^{-1} (y - \Phi v).$$

Solution

The projection of $v \in \mathbb{R}^n$ onto the set

$$C = \{x \in \mathbb{R}^n : \Phi x = y\}$$

is defined as the solution of

$$\Pi_C(v) = \arg \min_{x \in C} \frac{1}{2} \|x - v\|_2^2.$$

Therefore, we need to solve

$$\min_x \frac{1}{2} \|x - v\|_2^2$$

subject to

$$\Phi x = y.$$

Define the Lagrangian as,

$$L(x, \mu) = \frac{1}{2} \|x - v\|_2^2 + \mu^\top (\Phi x - y),$$

where $\mu \in \mathbb{R}^m$ is the vector of Lagrange multipliers.

For the minimizer, the gradient of the Lagrangian with respect to x must be zero. Thus,

$$\nabla_x L(x, \mu) = x - v + \Phi^\top \mu = 0.$$

Hence,

$$x = v - \Phi^\top \mu.$$

Now, the minimizer must satisfy the constraint $\Phi x = y$. Substituting the above expression for x into the constraint gives

$$\Phi(v - \Phi^\top \mu) = y.$$

Therefore,

$$\Phi v - \Phi \Phi^\top \mu = y.$$

Rearranging,

$$\Phi\Phi^\top\mu = \Phi v - y.$$

Since Φ has full row rank, $\Phi\Phi^\top$ is invertible. Hence,

$$\mu = (\Phi\Phi^\top)^{-1}(\Phi v - y).$$

Substituting this expression for μ into

$$x = v - \Phi^\top\mu,$$

we obtain

$$x = v - \Phi^\top(\Phi\Phi^\top)^{-1}(\Phi v - y).$$

Rearranging the terms gives

$$x = v + \Phi^\top(\Phi\Phi^\top)^{-1}(y - \Phi v).$$

Since this x is the unique minimizer of the projection problem, we have

$$\boxed{\Pi_C(v) = v + \Phi^\top(\Phi\Phi^\top)^{-1}(y - \Phi v)}.$$

□

Gen-AI Usage

ChatGPT was used for the following tasks in this assignment:

- Generating the LaTeX template and help with the LaTeX syntax for the submission. (It was not used to solve the problems)
- Writing the plotting syntax for codes in Q5.

References

- Sparsity Methods for Systems and Control
- CVX documentation [link](#)

The images and the code used in this submission can be accessed through [GitHub](#)